Final Project Sprint 3:
CS 6440

Lana P kareem, Valentina Cano Arcay, Faezeh Goli
lkareem6@gatech.edu, varcay3@gatech.edu, faezeh@gatech.edu

1 ACCOMPLISHEMENTS

By the previous Sprint (Sprint 2) due date, the scope of our project was finalized with our mentor, and the sprint was delivered by the deadline. The submission included details about our technical design, required APIs and data sources, architecture diagram, screen mock-ups, and implementation plan with assigned tasks and timelines. We also developed a detailed project timeline outlining each step and elaborated on potential risks that might arise during the project.

Since the last project check-in, each team member has been assigned specific tasks to complete over the past two weeks, from October 13 to the Sprint 3 deadline on October 27.

Below is a detailed list of the tasks completed by each group member, along with the estimated number of hours each member spent per week since the last check-in.

1.1 Faezeh

My plan for two weeks, ending by the Sprint 3 deadline, is to verify availability and documentation for all the APIs and data sources we will need for our group project.
I will also create connection examples and add code snippets to my reports, before sharing them with my teammates.
Any identified challenges in integration and also the solution I might suggest, will also be attached.
For the first week, I want to make sure we fully understand documentation and

check access requirements and endpoints for each API. It is also crucial to know the syntaxes for connections and what data each data source would provide.
The breakdown of my work for week 1 and 2, leading to Sprint 3 deadline is as follows:

Week 1:

- Fast API [1] and PubMed [2] (2 hours)
    - Reviewing the Fast API documentation
    - Searching the PubMed API
- RxNorm API [3] (2 hours)
    - Reviewing RxNorm documentation
    - Note on required endpoints to support autocomplete and normalization
- OpenFDA API [4] (2 hours)
    - Reviewing OpenFDA drug event API documentation
    - Note on parameters, and filtering
- FHIR API [5] (2 hours)
    - Reviewing FHIR R4 resources.
        - Observation, Condition, MedicationRequest
    - Understand JSON response structure.
- Summarizing the notes (1 hour)
- Creating a short document (1 hour)
    - Creating a document including the summary of notes taken, from reviewing documentations, including the API's base URL, endpoints, and response format.
    - Identifying the missing connections between APIs.

Week 2:

- OMOP CDM [6] (2 hours)
    - Reviewing OMOP documentation

    - Mapping OMOP columns to FHIR
- DrugBank integration [7] (2 hours)
    - Exploring DrugBank API
    - Identify if DrugBank open data, covers interaction
- SQLAlchemy [8] (1 hour)
    - Exploring a small model and investigate how to create dummy data
- Listing challenges we might face during our implementation (1 hour)
- Exploring the next required steps for UI design (1 hour)
- Adding second week notes to the report (1 hour)
- Preparing the Sprint 3 draft and share with the team (1 hour)
- Adding my tasks breakdown to the Sprint draft (1 hour)

1.2 Lana

My goal for this sprint was to establish the backend environment and create the initial project skeleton. In week 1 of 2 of this sprint, I made the following progress:

- Structured the repository with separate frontend and backend directories.
- Setup the backend:
    - Created the app/main.py (where the core functionality will be located) with root, /api/health, and /api/version endpoints.
    - Added CORS middleware to allow future frontend requests from http://localhost:5173.
    - Defined environment variables in the .env file, with variables such as APP_NAME, API_PREFIX, etc.
    - Setup the dependencies for the backend in requirements.txt
- Locally tested the backend
    - When running FastAPI backend and navigating to http://localhost:8000/docs in my browser, FastAPI automatically provides an interactive page to monitor API endpoints (called Swagger UI)

        - This page lists all the API endpoints (such as /api/health)
        - Tests endpoints in the browser without needing curl or Postman
        - Once more routes are added, they will automatically show on this page
    - Confirmed that /api/health endpoint returns STATUS OK.
    - Swagger docs located at /docs for future API development.

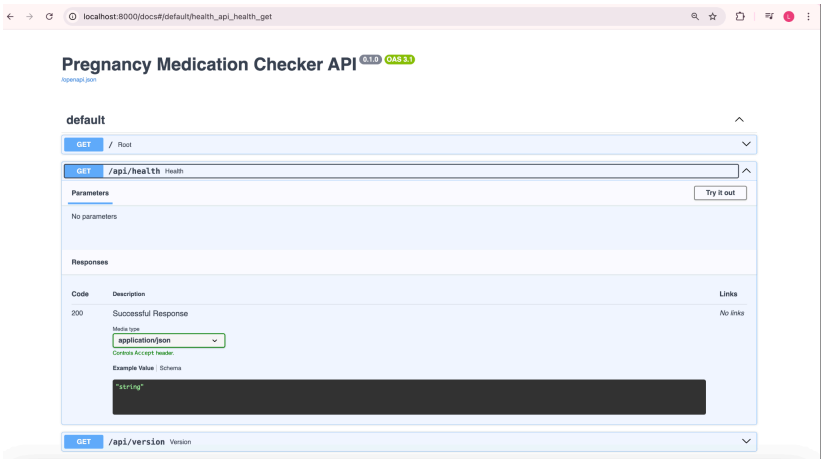

Figure 1—Confirmation of /api/health endpoint returning proper response.

- Setup Docker for containerization
    - Created a first draft of Dockerfile.backend, Dockerfile.frontend, and docker-compose.yml.

For week 1, here is how my time was divided for the above completed tasks:

- 3 hours: environment setup (venv, dependencies, gitignore, repo structure).
- 4 hours: writing and testing the main FastAPI app (main.py).
- 3 hours: Drafting Docker files for backend and frontend.

In week 2 of this sprint, my goal was to start setting up the frontend and try to connect it to the backend to have a blank UI to start with. Here is the progress I made with that:

- Frontend setup attempt with Vite (React + TS)
    - Command to set up the frontend (npm create vite@latest frontend -- --template react-ts) kept failing due to npm ETIMEDOUT errors when reaching the registry.
    - I investigated the above issue further by checking npm registry access with curl and was able to verify that no proxy was set, however npm still times out.
    - My next step to resolve this is turning off my VPN and trying to connect to a more stable internet.
- Frontend src code setup
    - The main files needed to set up the UI are prepared (main.tsx, App.tsx, index.html, package.json, tsconfig.json, vite.config.ts), but connection to the backend (/api/health endpoint) has not been verified yet in the browser.

The following is the time breakdown in week 2:

- 2 hours: npm/Vite troubleshooting (timeouts, proxies)
- 5 hours: preparing React frontend main files (main.tsx, App.tsx)
- 2 hours: setting up frontend dependencies (index.html, package.json, tsconfig.json, vite.config.ts)
- 1 hour: searching possible frontend alternatives (Next.js)

1.3 Valentina

My goal for these 2 weeks was to set up an initial connection to the FHIR server and ingest/search with some mock data.

Week 1 (6 hours)

- FHIR Client Library Research (2 hours): Investigated FHIR client libraries and FHIR R4 specifications to understand the API structure and best practices for server communication. Chose the https://github.com/smart-on-fhir/client-py in Python since I have more experience in this language

- Data Model Investigation (1.5 hours): Explored fhir.resources vs fhirclient.models libraries and identified where Pydantic models would be useful for API output validation
- Project and Architecture Setup (2.5 hours): Created Makefile for task automation, pyproject.toml with uv dependency management for Python, and created a fhir_integration folder within the backend part with interfaces/services/api separation (decided on this architecture for cleaner code, instead of having just one level for all classes). Created a README with some instructions on how to use the different make commands, so we can have consistency in development environments and standard formatting using lint and format. Refreshed my knowledge on Docker and added to the backend Dockerfile

Week 2 (13 hours)

- FHIR Service Implementation (4 hours): Built FHIRClient class with connection testing, patient search, and resource creation methods. Implemented error handling using Loguru for structured logging, added Python type hints. Created Pydantic data models and constants/enums to use throughout for better type-checking, error minimization, and cleaner code.
- Data Ingestion System (3 hours): Created DataIngestion class with 5 realistic pregnancy-focused test patients, common pregnancy medications, pregnancy-related conditions, and pregnancy observations. Used JSON test data files for easy data management
- API Integration (2 hours): Integrated FHIR services with FastAPI endpoints using clean handler pattern, creating various FHIR-related API endpoints in main.py
- Testing and Debugging (3 hours): Created initial testing infrastructure, added test commands to Makefile, handled server data quality issues from public FHIR by adding fallback search mechanisms using direct HTTP requests when fhirclient validation fails, and fixed all error that came up after running make format and lint checks

- Sprint Document (1 hour): Went over the tasks of these 2 weeks and completed my portion of this document, which was set up by Faezeh

2 CHALLENGES

Below are the challenges each member of our group faced, or anticipated they might face, during their work and preparation for Sprint 3, organized by individual team members.

2.1 Faezeh

I categorized the challenges I identified during my tasks by week.

Week 1 challenges:

- Exploring FastAPI, I found e parameters quite challenging to understand and manage.
- RXNorm responses are complicated nested JSON.
- There is a query limit, of 100 records max per page in OpenFDA Drug API, for interpreting adverse events, which is important to note.

Week 2 challenges:

- Mapping OMOP columns to FHIR resources, large datasets add complexity.
- Licensing is required for full access to DrugBank API.
- In completing my reports for my task, I found that in large documents, keeping notes concise and clear while summarizing them is quite a challenge. I try to maximize readability and keep the content short and to the point at the same time.

2.2 Lana

The challenges I faced were mainly in week 2 with setting up the frontend, where I experienced time out errors when trying to run npm install. Creating the React app with Vite kept failing due to connectivity issues, and despite my debugging, I was not able to resolve this issue within the second week. My next attempt will involve turning off my VPN, as I found that this could be a potential blocker.

This will likely reset the template code that I have created in the frontend folder in the repository, so in the upcoming sprint I will need to recreate the files within that folder. I plan to spend more time learning the details of Vite + React.js to ensure I can implement the frontend correctly once the repository setup is stable.

In week 1, I did not face any blockers, but rather did but put in a good amount of effort in configuring the CORS correctly so that the frontend would be allowed to make requests later.

2.3 Valentina

- FHIR resources: When testing I had conflicts between fhir.resources and fhirclient.models libraries. I tried to use fhir.resources because it has comprehensive FHIR R4 models with built-in validation, but since fhirclient.models is specifically designed for SMART on FHIR server, I realized it was a better fit for server operations. I then decided to also create some custom Pydantic models for API output validation
- Task Automation Tool Conflicts: Tried to use Taskfile (which I use professionally) but had conflicts with Taskwarrior both using the task command. After trying to resolve conflicts with aliases, I switched to Makefile which is simpler, more universally available, and avoids tool conflicts for better team collaboration
- Public FHIR Server Data Quality Issues: the HAPI FHIR server had malformed data errors ("Narrative objects missing div properties") and Firely rejected the patient data with 400 Bad Request errors. In the end I kept HAPI but since search operations frequently failed, I had to implement a fallback to direct HTTP requests. For next sprint, I need to investigate these issues further and explore generating better synthetic data.

3 SPRINT PLANS

3.1 Plans for the next 2 weeks

The group's plan for the next two weeks, until the next project check-in due date on November 10, is to implement the core interaction checker logic. This includes developing the frontend, connecting to the RxNorm API, and testing the overall functionality. On a more granular level, the team will implement the medication input form (frontend) and the backend endpoint to process the medication list, which involves connecting to the RxNorm API for medication normalization and autocomplete suggestions. The interaction checker logic will also be implemented using dummy data. Additionally, the team plans to integrate the required APIs, such as DrugBank for drug interaction data and PubMed for real-time research searches. We will continue to implement the FHIR API for patient data retrieval, which will be enhanced with better error handling and data validation, along with research into Synthea synthetic data generation to replace hard-coded test data with realistic and useful patient datasets. We will also implement the search functions for medications, observations, and conditions in the FHIR server.

4 REFERENCES

1. Tiangolo. (n.d.). *FastAPI documentation*. Retrieved October 22, 2025, from https://fastapi.tiangolo.com/
2. National Center for Biotechnology Information. (n.d.). *PubMed help*. Retrieved October 22, 2025, from https://pubmed.ncbi.nlm.nih.gov/help/
3. National Library of Medicine. (2025, January 7). *RxNorm overview*. U.S. National Library of Medicine. https://www.nlm.nih.gov/research/umls/rxnorm/overview.html
4. U.S. Food & Drug Administration. (n.d.). *How to use the endpoint – Drugs@FDA (openFDA API)*. Retrieved October 22, 2025, from https://open.fda.gov/apis/drug/drugsfda/how-to-use-the-endpoint/

5. Health Level Seven International. (n.d.). *FHIR RESTful API (FHIR® specification)*. Retrieved October 22, 2025, https://build.fhir.org/http.html
6. Observational Health Data Sciences and Informatics. (n.d.). *OMOP Common Data Model (CDM)*. Retrieved October 22, 2025, from https://ohdsi.github.io/CommonDataModel/
7. DrugBank Online. (n.d.). *About DrugBank Online*. Retrieved October 22, 2025, from https://go.drugbank.com/about
8. SQLAlchemy authors. (n.d.). *SQLAlchemy: The Python SQL toolkit and object–relational mapper*. Retrieved October 22, 2025, from https://www.sqlalchemy.org/